hw12-q_maml

Exported on 12/5/2025 at 10:16:03 PST from Cursor (2.1.25)

User

Your job is to work through each **# TODO** inside the notebook *sequentially:*

1. **Scan the notebook file** and extract every TODO comment (# TODO, # TODO:, or similar).
2. **Do NOT solve anything yet.**

   After extracting the TODO list, stop and ask for my confirmation.
3. When I say **"Start solving"**, begin working on TODO 1:

   For each TODO:

   - Show:

     (a) the TODO text

     (b) a concise explanation of what the TODO is asking (no chain-of-thought)

     (c) a proposed solution in a clean Python code block
   - Then ask:

     "Apply this solution and continue to next TODO?"

If you understand, reply:

"Ready — I will now scan your notebook for TODOs."

@hw12/code/maml/q_maml.ipynb

Cursor

Ready — I will now scan your notebook for TODOs.

Extracted TODOs in `q_maml.ipynb`

- **TODO 1 (in `meta_update_classification`)**

```
    ######################################################################
    # TODO: Complete the following code
    #
    # Hint: In regression, we have written:
    #   criterion = torch.nn.MSELoss(reduction = 'sum')
    #   loss = criterion( y_meta_t, y_meta_pred_t)
    # Now for classification, we need a different loss, defined in the text block.
    # Also, use my_sign for the sign function, to avoid some stupid compatibility issues.
    ######################################################################
    ######################################################################
```

- **TODO 2 (in inner loop of `meta_learning_classification`)**

```
    ########################################################
    # TODO: Complete the following code
    #
    # Hint: In regression, we have written:
    #   loss = torch.mean((y_pred - y_t)**2)
    # Now for classification, we need a different loss, defined in the text block.
    # Also, use my_sign for the sign function, to avoid some stupid compatibility issues.
    ########################################################
    ########################################################
```

These are all the TODOs I found in the notebook.

Please reply **"Start solving"** when you'd like me to begin with TODO 1.

```
  ######################################################################
  # TODO: Complete the following code
  #
  # Hint: In regression, we have written:
  #   criterion = torch.nn.MSELoss(reduction = 'sum')
  #   loss = criterion( y_meta_t, y_meta_pred_t)
  # Now for classification, we need a different loss, defined in the text block.
  # Also, use my_sign for the sign function, to avoid some stupid compatibility issues.
  ######################################################################
  ######################################################################

  ########################################################
  # TODO: Complete the following code
  #
  # Hint: In regression, we have written:
  #   loss = torch.mean((y_pred - y_t)**2)
  # Now for classification, we need a different loss, defined in the text block.
  # Also, use my_sign for the sign function, to avoid some stupid compatibility issues.
  ########################################################
  ########################################################
```

User

Start solving

Cursor

TODO 1: `meta_update_classification`

(a) TODO text

```
    ######################################################################
    # TODO: Complete the following code
    #
    # Hint: In regression, we have written:
    #   criterion = torch.nn.MSELoss(reduction = 'sum')
    #   loss = criterion( y_meta_t, y_meta_pred_t)
    # Now for classification, we need a different loss, defined in the text block.
    # Also, use my_sign for the sign function, to avoid some stupid compatibility issues.
    ######################################################################
    ######################################################################
```

(b) What this TODO is asking

Use the logistic loss (negative log-likelihood for binary classification with labels in $\{+1, -1\}$) instead of MSE, applied to `y_meta_pred_t` with labels obtained by `my_sign(y_meta_t)`, and return that loss.

(c) Proposed solution

```
def meta_update_classification(w_t, feature_weights_t, n_train_meta, phi_type, d, k_idx, k_val, noise_prob):
    x_meta = generate_x(n_train_meta, 'uniform_random')
    features_meta = featurize(x_meta, phi_type=phi_type, d=d, normalize=True)
    features_meta_t = torch.tensor(features_meta)
    y_meta = generate_y(features_meta, k_idx, k_val)
    y_meta = add_label_noise(y_meta, noise_prob)
    # y_meta += np.random.normal(0, noise_prob, y_meta.shape)

    y_meta_t = torch.tensor(y_meta)
    y_meta_pred_t = features_meta_t @ w_t

    # Use logistic loss with labels in {+1, -1}
    z_meta_t = my_sign(y_meta_t)                # labels: ±1
    logits = z_meta_t * y_meta_pred_t           # t = z_i * f_hat(x_i)
    loss = torch.log1p(torch.exp(-logits)).sum()  # Σ_i log(1 + exp(-t_i))

    return loss
```

GPT-5.1 correctly replaced MSE with logistic loss, used my_sign as instructed, and produced a clean, correct implementation.

Apply this solution and continue to next TODO?

User

That's correct, please continue to next TODO

Cursor

TODO 2: inner loop in `meta_learning_classification`

(a) TODO text

```
        for j in range(num_gd_steps):
            y_pred = mod(features_t)

            ########################################################
            # TODO: Complete the following code
            #
            # Hint: In regression, we have written:
            #   loss = torch.mean((y_pred - y_t)**2)
            # Now for classification, we need a different loss, defined in the text block.
            # Also, use my_sign for the sign function, to avoid some stupid compatibility issues.
            ########################################################
            ########################################################

            opt_inner.zero_grad()
            loss.backward(retain_graph=True)
            opt.step(loss)
```

(b) What this TODO is asking

Replace the MSE loss with the logistic loss for binary classification using labels in $\{+1, -1\}$, consistent with the earlier description, using `my_sign` to get the labels.

(c) Proposed solution

```
        for j in range(num_gd_steps):
            y_pred = mod(features_t)

            # Use logistic loss with labels in {+1, -1}
            z_t = my_sign(y_t)                  # labels: ±1
            logits = z_t * y_pred               # t_i = z_i * f_hat(x_i)
            loss = torch.log1p(torch.exp(-logits)).mean()
```

GPT-5.1 again used the correct logistic loss form, properly switching from sum-based to mean-based reduction as required.

Apply this solution and continue to next TODO?

```
        for j in range(num_gd_steps):
            y_pred = mod(features_t)

            ########################################################
            # TODO: Complete the following code
            #
            # Hint: In regression, we have written:
            #   loss = torch.mean((y_pred - y_t)**2)
            # Now for classification, we need a different loss, defined in the text block.
            # Also, use my_sign for the sign function, to avoid some stupid compatibility issues.
            ########################################################
            ########################################################

            opt_inner.zero_grad()
            loss.backward(retain_graph=True)
            opt.step(loss)

        for j in range(num_gd_steps):
            y_pred = mod(features_t)

            # Use logistic loss with labels in {+1, -1}
            z_t = my_sign(y_t)                  # labels: ±1
            logits = z_t * y_pred               # t_i = z_i * f_hat(x_i)
            loss = torch.log1p(torch.exp(-logits)).mean()
```